International Journal on Artificial Intelligence Tools
Vol. 32, No. 6 (2023) 2350017 (26 pages)
© World Scientific Publishing Company
DOI: 10.1142/S0218213023500173

World Scientific
www.worldscientific.com

Annotated OpenAPI Descriptions and Ontology for REST Services

Nikolaos Mainas*, Fotios Bouraimis†, Aikaterini Karavisileiou‡
Euripides G. M. Petrakis§

School of Electrical and Computer Engineering
Technical University of Crete (TUC), Chania, Crete, Greece
**nmainas@tuc.gr*
†*fbouraimis@tuc.gr*
‡*akaravasileiou@tuc.gr*
§*petrakis@intelligence.tuc.gr*

Received 25 October 2021
Accepted 26 October 2022
Published 25 September 2023

Leveraging the latest research results on hypermedia-driven Web APIs and the newest update of the OpenAPI Specification, we propose a reference ontology for REST services along with a formal procedure for converting OpenAPI service descriptions to instances of this ontology. At the heart of the approach is a model for enhancing the meaning of Schema properties (i.e. re-usable JSON Schema properties that clarify the meaning of service components). Schema properties are semantically annotated (i.e. their meaning is mapped to a semantic model) and existing properties are combined to form composed or polymorphic expressions. The algorithm for mapping service descriptions to the OpenAPI ontology is implemented and is available as a Web Application.

Keywords: Web service; ontology; OpenAPI; REST; Hydra; SHACL.

1. Introduction

Publishing service descriptions on the Web is critical to their discovery and market success. The OpenAPI Specification[a] is a powerful framework for describing REST services. It is a widely used standard endorsed by the Linux Foundation and supported by major software vendors such as Google and Microsoft.

OpenAPI comprises a large set of properties for composing service descriptions. The syntactic binding of OpenAPI format to JSON (or YAML) complicates the detection of similarities, inconsistencies, or ambiguities in service descriptions. In a previous work[1] we analyzed the causes of ambiguity and proposed a reference

§Corresponding author
[a]https://www.openapis.org

ontology for OpenAPI descriptions based on Hydra.[2] The OpenAPI ontology incorporates the majority of Hydra concepts for modeling service operations and adds new models for REST security, headers and constraints. Classes together with constraints on class properties are described using SHACL[3] allowing service descriptions to be validated against the ontology. The ontology instantiation algorithm[4] handles all OpenAPI objects and their properties (e.g. paths, operations, and reusable objects such as security and schemas). At the heart of the approach is a model for enhancing the meaning of *Schema* properties. *Schema* objects are semantically annotated (i.e. their meaning is mapped to a semantic model) and existing properties can be combined to form complex composed or polymorphic expressions.

This paper summarizes, extends, and integrates the authors' previous work efforts and proposes (a) an OpenAPI annotation mechanism to eliminate ambiguities in service descriptions, (b) an ontology for REST services, (c) exploiting SHACL for validating OpenAPI descriptions against the ontology, and (d) an algorithm that maps OpenAPI descriptions to the ontology. The Web application[b] supports converting OpenAPI descriptions into instances of the OpenAPI ontology, downloading the ontology, and searching for services that meet specific criteria using SPARQL.

Related work on services description languages (with emphasis on REST services) is presented in Sec. 2. OpenAPI specification and the annotation method for resolving ambiguities in service descriptions are discussed in Sec. 3. The OpenAPI ontology and the instantiation of service descriptions to the ontology are discussed in Sec. 4. Ontology evaluation is described in Sec. 5 followed by issues for future work in Sec. 6.

2. Related Work

Syntactic description languages describe the requirements for establishing a connection with a service and the message formats to successfully communicate with it. WSDL[5] is an XML-based interface description language for SOAP services[6] (i.e. how they can be invoked). The latest version 2.0 introduced changes in the document structure as well as HTPP 1.1 protocol support to allow the description of REST services. SAWSDL[7] defined extension properties to add semantics to WSDL components. WADL[8] is yet another XML-based interface description language designed to describe REST services. It is closely related to WSDL; it attempts to model the resources provided by the service and their relationships but has limited support for describing the meaning of service resources. RAML[c] is compatible with common machine-readable formats like XML and JSON.

Semantic approaches describe services by means of semantic models (i.e. ontologies) and are more capable of supporting automated service discovery and composition. OWL-S[9] is an upper ontology for Web services but, similar to all

[b]https://www.intelligence.tuc.gr/semantic-open-api/
[c]https://raml.org

other methods, does not support the dynamic discovery of resources at runtime. SA-REST[10] offers a set of properties for annotating service descriptions in HTML. Hydra[2] simplifies the construction of hypermedia-driven APIs. The Hydra vocabulary defines concepts that a server can use to advertise valid state transitions to a client as a result of a sequence of service invocations.

Musyaffa *et al.*[11] introduce annotations in *Schema* and *Parameter* objects for OpenAPI v2.0. They do not handle all causes of ambiguity. The annotations appear within text properties and cannot be interpreted by a machine without pre-processing. Schwichtenberg, Gerth, and Engels[12] map OpenAPI v2.0 service descriptions to OWL-S ontology but do not deal with any causes of ambiguity, nor do they handle security properties. The mapping is error-prone and must be adjusted manually. Their choice of OWL-S for representing REST services is controversial: OWL-S is suitable for SOAP services but not for hypermedia-driven Web services like REST. Both methods, do not handle OpenAPI 3.0 descriptions.

3. OpenAPI

OpenAPI service descriptions comprise many objects.[d] Each object has a list of properties that can be objects as well. Objects and properties defined under the *Components* unit of an OpenAPI document can be re-used by other objects or they can be linked to each other (e.g. using keyword *$ref*). However, these links are not always explicitly expressed (e.g. there are properties with the same name with no reference to one another or to an external model).

The *Info* object provides non-functional information such as the name of the service, service provider, and terms of use. The *Servers* object describes where the API servers are located (i.e. multiple servers can be defined). The *Security* object describes the security schemes that the service uses for authentication (i.e. API keys, OAuth2.0 common flows and OpenID Connect). The *Paths* object contains the relative paths of the service endpoints. Each *Path* item describes the available operations based on HTTP methods (e.g. get, put, post).

The core of an OpenAPI document is the *Operation* object that provides information for expressing HTTP requests to the service and its responses. The *Components* object describes a set of reusable objects. These objects can be responses, parameters, schemas, request bodies, and more. The *Responses* object specifies the expected responses of an *Operation* and maps each response to an HTTP status code and to any HTTP Headers that an operation's response may return. The *Parameters* object specifies the parameters that operations can use. These can be path parameters (i.e. specified in the operation's path), query parameters (which are appended to the URL when sending a request), header, or cookie parameters.

The *Schema* object specifies the data type that describes the request and response messages. It can be a primitive (i.e. string, integer), an array, or a model.

[d]https://blog.readme.io/an-example-filled-guide-to-swagger-3-2/

For the definition of *Schema* objects, the specification resorts to JSON Schema.[e] A *Schema* object definition can be enhanced with XML data types. New data types can be defined as a combination of existing ones (i.e. using the *allOf* property or, the *oneOf* property).

3.1. *Semantically annotated OpenAPI*

Despite its rigorous service language format (JSON or YAML), OpenAPI service descriptions can be vague. There can be properties that share the same meaning (although they are defined using different names) or, their meaning is ambiguous. Probably, a human can easily resolve ambiguities either by the element names or by the description that may be provided with the properties. For a machine to act similarly to a human, it is necessary to provide additional information. To eliminate ambiguities, OpenAPI properties must be semantically annotated and associated with entities of a semantic model (e.g. using *Schema.org* vocabulary) using the extension properties of Table 1.

The *x-refersTo* extension property specifies the association between an OpenAPI element and a concept in a semantic model. The *x-refersTo* property in Listing 1 is used to annotate a *Pet* model and its properties: it associates the model with *Pet* class in *Product* ontology.[f] Listing 1 demonstrates how polymorphism is expressed using the discriminator property: the model of a *Pet* uses the *name* property as a discriminator to determine whether this is a model of a *dog* or of a *cat*. The *allOf* property is used to specify that the model of a *dog* or a *cat* has the properties of a *Pet* model and also additional properties. If a model describes a specific group

Table 1. OpenAPI extension properties.

Property	Applies to	Meaning
x-refersTo	Schema object	The concept in a semantic model that describes an OpenAPI element.
x-kindOf	Schema object	A specialization between an OpenAPI element and a concept in a semantic model.
x-mapsTo	Schema object	An OpenAPI element which is semantically similar with another OpenAPI element.
x-collectionOn	Schema object	A model describes a collection over a specific property.
x-onResource	Tag object	The specific *Tag* object refers to a resource described by a Schema object.
x-operationType	Operation object	Clarifies the type of operation.

[e] http://json-schema.org
[f] http://www.productontology.org/

Listing 1. OpenAPI model polymorphism example.

```
parameters:
 Query:
  name: name
  in: query
  required: true
  schema:
   type: string
   x-mapsTo: '#/components/schemas/Pet.name'
schemas:
 Pet: # A Pet model extended with Annoated OpenAPI v3.0 properties
  type: object
   x-refersTo: http://www.productontology.org/doc/Pet
   properties:
    name:
     type: string
     x-refersTo: http://purl.org/goodrelations/v1#name
    photoUrls:
     type: string
     x-refersTo: https://schema.org/image
    id:
     type: integer
     x-refersTo: https://schema.org/identifier
    required:
     - name
     - photoUrls
    discriminator:
     propertyName: name
 Dog: # A Dog model extending the Pet Model
  x-kindOf: http://www.productontology.org/doc/Pet
  allOf:
   - $ref: '#/components/schemas/Pet'
   - type: object
    properties:
     breed:
      type: string
    required:
     - breed
 Cat: # A Cat model extending the Pet Model
  x-kindOf: http://www.productontology.org/doc/Pet
  allOf:
   - $ref: '#/components/schemas/Pet'
  ...
```

of pets (e.g. dogs), the *x-kindOf* property is used to denote that the model is a subclass of the referred semantic concept.

The *x-refersTo* and *x-kindOf* link OpenAPI elements with concepts from a semantic model. The *x-mapsTo* is used to semantically associate an OpenAPI element with another OpenAPI element within the same OpenAPI document. The OpenAPI element using the *x-mapsTo* property is considered to be semantically similar to the OpenAPI element it refers to. In Listing 1 the *x-mapsTo* property is used in *Parameters* object to dictate that query parameter *name* refers to the property *name* of *Schema* object *Pet*. There is a semantic similarity between the *id* property of *Pet* model and the *petId* property in *https://schema.org/identifier* which later appears in the document as *petId* (not shown in the example). Similarly, *Schema* object *Pet* should refers to *http://www.productontology.org/doc/Pet* and property *name* refers to *http://purl.org/goodrelations/v1#name*.

The *x-collectionOn* property indicates that a model in *Schemas* object is a collection. Often, a collection (or a list) of resources is described using the *array* type. It is common for a collection's definition to be encapsulated within an object type with additional properties. The *x-collectionOn* property is used to denote the data

Listing 2. Model definition representing a collection.

```
schemas:
 PetCollection:  # A Pet Collection definition
  x-collectionOn: pets
  type: object
  properties:
   pets:
    type: array
    items:
     $ref: '#/components/schemas/Pet'
   totalItems:
    type: integer
```

Listing 3. Associating Schema objects with a Tag.

```
tags:
 name: pet
 description: Everything about your Pets
 externalDocs:
  description: Find out more
  url: 'http://swagger.io'
 x-onResource: '#/components/schemas/Pet'

paths:
 /pet/findByStatus:
  get:
   x-operationType: 'http://schema.org/SearchAction'
  tags:
   - pet
 summary: Finds Pets by status
 description: Multiple status values can be provided
 operationId: findPetsByStatus
 parameters:
  - $ref: '#/components/parameters/statusQuery'
 responses:
  '200':
   description: successful operation
   content:
    application/json:
   schema:
    $ref: "#/components/schemas/Pets"
  '400':
   description: Invalid status value
 security:
  - petstore_auth:
  - 'write:pets'
  - 'read:pets'
```

types of the objects of the collection. Listing 2 defines a model as a collection of *Pet* objects (*totalItems* property denotes population).

The *x-onResource* extension property is used in *Tag* objects to specify the resource that a tag refers to. Tags are used to group operations either by resources or any other qualifier. If the tag is used to group operations by resources, a human may recognize that the referred resource is described by a *Schema* object in *Schemas* but a machine cannot. The *x-onResource* property is used to associate the tag with a *Schema* object that describes a specific resource. In Listing 3, the *x-onResource* property is assigned on a *Pet* tag that provides information about the operations that are available for *Pet* model in *Schemas* object.

The *x-operationType* extension property is used to specify the type of an *Operation* object. A request is characterized by an HTTP method but, the semantics of the HTTP methods are too generic and may have a more specific meaning. In Listing 4, the *x-operationType* extension property is used to clarify that a GET

Listing 4. Clarifying the meaning of an operation.

```
paths:
 /pet/findByStatus:
 get:
  x-operationType: 'http://schema.org/SearchAction'
  tags:
   - pet
  summary: Finds Pets by status
  description: Multiple status values can be provided with comma
      separated strings
  operationId: findPetsByStatus
  produces:
   - application/xml
   - application/json
  parameters:
   - $ref: '#/parameters/statusQuery'
  responses:
  '200':
   description: successful operation
   schema:
    $ref: '#/definitions/PetCollection'
  '400':
   description: Invalid status value
  security:
   - petstore_auth:
    - 'write:pets'
    - 'read:pets'
```

request on path */pet/findByStatus* is a search operation on pets based on their status. The value of the property is a URL pointing to the concept that semantically describes the operation type. The *Action*[g] type of the *Schema.org* vocabulary provides a detailed hierarchy of *Action* sub-types that can be used by the property.

4. OpenAPI Ontology

The semantic meaning of the service is captured by the OpenAPI ontology of Fig. 1. At the heart of the ontology is *Hydra* core vocabulary.

Document class represents the documentation and general information (i.e. *Info* class) regarding the service, specifying the service paths and the entities that it supports. The Path class represents (relative) service paths (i.e. property *pathName*). Class Operation provides information for sending requests. Request bodies are represented by the Body class and responses are declared in the Response class specifying the status code and the data returned. The MediaType class describes the format of a request or response body data (the most common being JSON and XML). Class Operation refers to a security scheme in SecurityRequirement class.

OpenAPI parameters are separate classes for every parameter type. The Header class provides all the definitions of header parameters. The Cookie class defines the cookies that are sent with HTTP requests. The Parameter class defines all parameters that are attached to the operations' URL. The class has PathParameter and Query sub-classes that refer to the corresponding path and query parameters of the specification. Request and response bodies, are defined as classes and so is defined their media type. Class Encoding defines keywords denoting serialization

[g]http://schema.org/Action

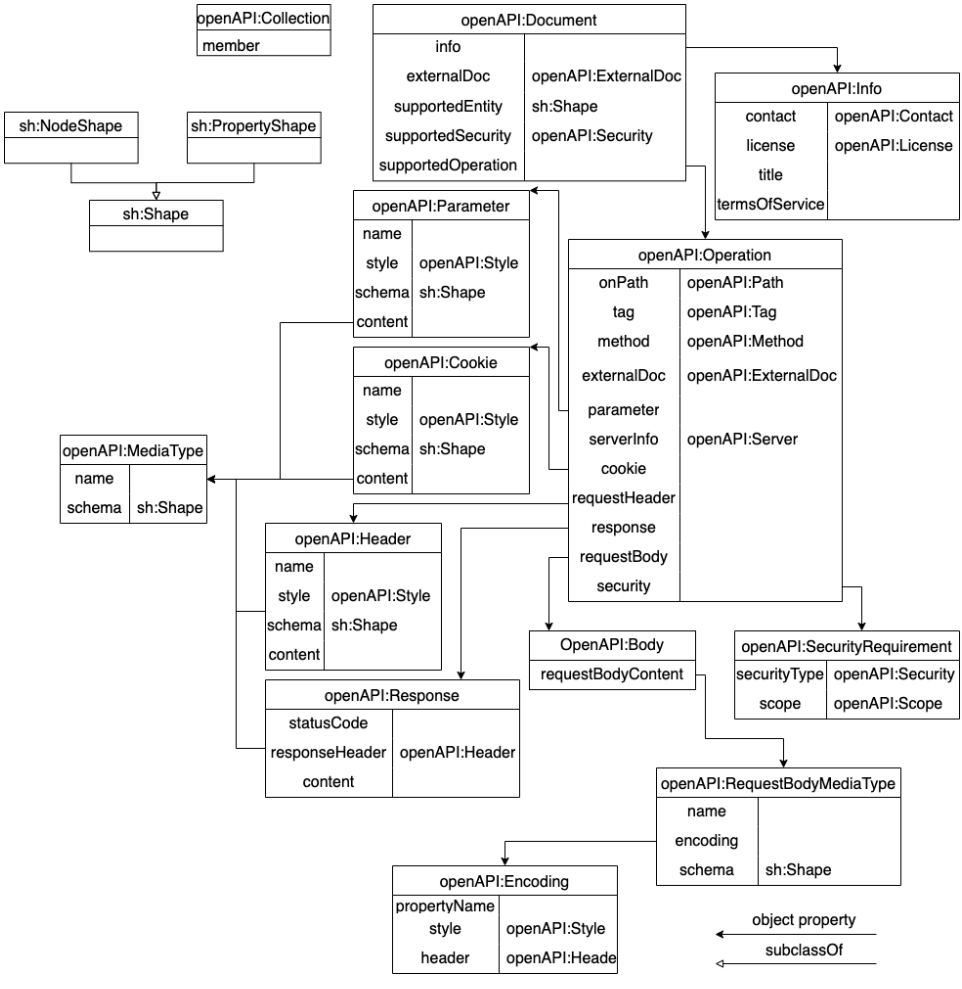


Fig. 1. OpenAPI 3.0 ontology.

rules for media types with primitive properties (e.g. *contentType* for nested arrays or JSON). Figure 2 shows the security schemes supported by OpenAPI. Class *Security* has security schemes as sub-classes. Class *OAuth2* has different flows (grants) as sub-classes. If the security scheme is of type *OAuth2* or *OpenID Connect*, then scope names are defined as properties.

4.1. *Instantiating OpenAPI service descriptions to the ontology*

Algorithm 1 summarizes the ontology instantiation process.[4] The algorithm scans the OpenAPI document and instantiates OpenAPI objects to ontology classes. Objects *Info*, *servers*, *Security schemes*, *Tags*, and *Path* become individuals of their corresponding classes.

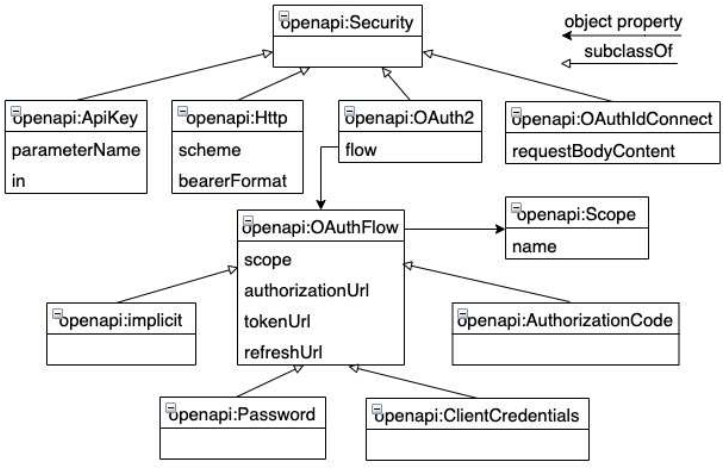


Fig. 2. OpenAPI 3.0 security class.

Initially, an individual class Document is created and *OpenAPI* object is mapped to class Document. Function *parseInfoObject* creates an instance of the Info class from information in *Contact* and *License* objects which, in turn, are mapped to individuals of their corresponding classes. Listing 5 illustrates the mapping of the *Info* object to the ontology. The next step is the mapping of *Server* objects to individuals of the Server class (lines 5–8). If there are more than one occurrence of servers in an OpenAPI file, they are stored in a list (line 5). The list will be overwritten by server information defined in *Path* or *Operation* objects. Security schemes are converted to individuals of the respective Security classes (Fig. 2) using method *parseSecuritySchemeObject* of Algorithm 2. A *Security scheme* object becomes an individual of a class (i.e. ApiKEY, HTTP, OpenIDConnect, or OAuth2) depending on the property type specified by the object. The security individual is connected to the Document individual with property *supportedSecurity*. Handling *Security Requirement* objects (lines 11–14) is similar to that of *Server* objects. *Security Requirement* objects are converted to individuals and are kept in a list. They can be overwritten by *Security requirements* defined in *Operation* objects.

Algorithm 1 resumes by handling *Tag* objects (lines 15–19). The *x-onResource* property associates a *Tag* object with a *Schema* object. A map is created with pairs of *Tags* and *Schemas*. Function *parseTagObject* (Algorithm 3) creates a *Tag* individual with all its properties. It also handles the *x-onResource* property. If the property exists, it creates a Shape individual from the referred *Schema* object and returns the pair of *Tag-Shape* individual; otherwise, it returns only the *Tag* individual. The pair is added to the appropriate map that will be used during the instantiation of *Operation* objects.

Finally, Algorithm 1 handles *Path* objects (lines 20–29). For every *Path* object, a *Path* individual is created and the function *parseServerObject* instantiates any

Algorithm 1 Converting OpenAPI object to ontology

```
1: procedure parseDocumentObject(OpenAPI document)
2:     Init Ontology Model, docInd ← create Document individual
3:     infoInd = parseInfoObject(Info object)                    ▷ Info individual
4:     docInd add relation openapi:info with infoInd
5:     globalServerInds ← init List
6:     for Server object in servers property do                  ▷ from OpenAPI object
7:         serverInd = parseServerObject(Server object)          ▷ Server individual
8:         globalServerInds add serverInd
9:     for Security Scheme object in securitySchemes property do ▷ from Components object
10:        parseSecuritySchemeObject(Document individual, Security Scheme object)
11:    globalSecReqObjects ← init List
12:    for Security Requirement object in security property do ▷ from OpenAPI object
13:        secReqInd = parseSecurityReqObject(Security Requirement object) ▷ Security Requirement individual
14:        globalSecReqObjects add secReqInd
15:    tagShapeMap ← init Map                ▷ pairs of Tags and Shapes individuals
16:    schemaObjects ← schemas property      ▷ from Components object
17:    for Tag object in tags property do    ▷ from OpenAPI object
18:        Pair(Tag, Shape) = parseTagObject(Tag object, schemaObjects)
19:        tagShapeMap add Pair
20:    for Path object in Paths object do
21:        pathInd ← Create Path individual
22:        pathServerInds ← init List
23:        for Server object in servers property do              ▷ from Path Item object
24:            serverInd = parseServerObject(Server object)      ▷ Server individual
25:            pathServerInds add serverInd
26:        pathParameterObjects ← parameters property            ▷ from Path object
27:        Select serverIndList (pathServerInds if not empty, otherwise globalServerInds)
28:        for Operation object in Path Item object do
29:            parseOperationObject(docInd, pathInd, Operation object, tagShapeMap, pathParameterObjectss, serverIndList, globalSecReqObjects)
```

servers defined in the *Path* object. *Server* individuals are kept in a list that represents the servers used in the current path. If the property *Servers* is not set, the list is replaced by the *globalServerList* defined earlier. Then, the algorithm extracts all the parameters defined in the current path that will be used on the instantiation of *Operation* objects (line 29). An *Operation* object may overwrite a lot of the previously mapped OpenAPI objects such as servers, parameters, and security requirements.

Listing 5. Instantiating an OpenAPI document object.

```
...
ex:PetStoreDocument
a openapi:Document ;
openapi:info [
openapi:serviceTitle "Sample Pet Store App";
openapi:description "This is a sample server for a pet store." ;
openapi:termsOfService <http://example.com/terms/> ;
openapi:contact [
openapi:creator "API Support" ;
openapi:url <http://www.example.com/support> ;
openapi:email <support@example.com> .
] ;
openapi:license [
openapi:licenseName "Apache 2.0" ;
openapi:url <http://www.apache.org/licenses/LICENSE-2.0.html>
]
] ;
...
```

Algorithm 2 Create individuals of *Security* class

function parseSecuritySchemeObject(Individual document, Security Scheme object)
    type ← object get type
    if type is apiKey **then**
        secInd ← create individual of ApiKey Class
    else if type is http **then**
        secInd ← create individual of Http Class
    else if type is oauth2 **then**
        secInd ← create individual of OAuth2 Class
    else if type is openIdConnect **then**
        secInd ← create individual of OpenIDConnect Class
    document add relation openapi:supportedSecurity with secInd

Algorithm 3 Create individuals of *Tag* class

function parseTagObject(Tag object, schemaList) ▷ schemaList, List of Schema objects
    tagInd ← Create Tag individual
    if Tag object contains property x-onResource **then**
        schema ← from schemaList ▷ find using value of x-onResource
        shape = parseSchemaObject(schema name, schema, schemaList) ▷ Shape individual
        return Pair of tagInd and shape
    else
        return Pair of tagInd and NO Shape

Listing 6. Instantiating path and operation objects to the ontology.

```
1  ...
2  ex:path2
3  a openapi:Path ;
4  openapi:pathName "/pets/findByStatus" ;
5  ex:path2_op1
6  a openapi:Operation, schema:SearchAction ;
7  openapi:onPath ex:path2
8  openapi:method openapi:GET ;
9  openapi:tag ex:tag_pet ;
10 openapi:parameter ex:query_status ;
11 openapi:response [
12 openapi:statusCode 200 ;
13 openapi:content [
14 openapi:mediaTypeName "application/json" ;
15 openapi:schema ex:PetCollectionShape
16 ] ;
17 openapi:description "successful operation"
18 ] ;
19 openapi:response [
20 openapi:statusCode 400 ;
21 openapi:description "Invalid status value" .
22 ] ;
23 openapi:security [
24 openapi:securityType ex:petstore_oauth ;
25 opneapi:scope ex:read_pets, ex:write_pets
26 ] ;
27 openapi:name "findPetsByStatus" ;
28 openapi:summary "Finds Pets by Status" ;
29 openapi:description "Multiple status values  with comma seperated
       strings" .
30 ...
```

These OpenAPI objects are arguments of function *parseOperationObject*. Listing 6 shows the individuals of *Path* and *Operation* objects of Listing 4.

Function *parseOperationObject* (Algorithm 4) is responsible for the instantiation of *Operation* objects. The algorithm checks if the *x-operationType* property exists in the objects and uses its value to make the *Operation* individual also an individual of the referred class. Regarding *Tags*, the algorithm uses the *tagShapeMap* to retrieve the respective *Tag* individuals and assign them to the *Operation* individual. If the retrieved *Tag* is linked with a Shape (*Schema* object) then the *Operation* individual is added as a supported operation in the Shape individual. For *Security requirements* and *Servers*, the algorithm checks the respective properties of the operation object and if they do not have values, then it uses the *Server* and *Security requirement* list from the function's input (lines 13–28).

Operation's parameters are defined in function *parseParameterObject* (Algorithm 5) which is responsible to instantiate them to the corresponding classes based on their type (path, cookie, header, query). Functions *parseQueryParameterObject*, *parsePathParameterObject*, *parseCookieParameterObject*, and *parseHeaderParameterObject* share the same logic. In all functions, the respective individual is created, and a *Schema* object is extracted that will be converted to Shape through the function *parseSchemaObject*.

The last step of the *parseOperationObject* function is the instantiation of the *Request Body* and *Response* objects (Algorithm 6). Function *parseRequestBodyObject* is responsible for handling *Request Body* objects. After creating the *Body* individual, the function collects all *Media Type* objects and for each *Media Type* calls

Algorithm 4 Create individual of *Operation* class

```
1: function parseOperationObject(docInd, pathInd, Operation object,
   tagShapeMap, pathParameterObjects, serverInds, globalSecReqInds,
   schemaList)
2:     opInd ← Create Operation individual
3:     if Operation's property x-operationType has value then
4:         make opInd also individual of x-operationType referred class
5:     opInd add relation openapi:onPath with Path individual
6:     opInd add relation openapi:supportedOperation with Document individual
7:     for tag in tags property do                          ▷ from Operation object
8:         if tagShapeMap contains tag then
9:             pair ← from tagShapeMap
10:            opInd add relation openapi:tag with tagInd                ▷ from pair
11:            if pair has Shape then
12:                Shape add relation openapi:supportedOperation with opInd
13:    securityReqObjects ← security property               ▷ from Operation object
14:    if securityReqObjects is empty then
15:        for secReqInd in globalSecurityReqIndividuals do
16:            opInd add relation openapi:security with secReqInd
17:    else
18:        for Security Requirement object in securityReqObjects do
19:            secReqInd = parseSecurityReqObject(Security Requirement object)
20:            opInd add relation openapi:security with secReqInd
21:    serverObjects ← servers property                     ▷ from Operation object
22:    if serverObjects is empty then
23:        for servInd in serverInds do
24:            opInd add relation openapi:serverInfo with servInd
25:    else
26:        for Server object in serverObjects do
27:            servInd = parseServerObject(Server object)          ▷ Server individual
28:            opInd add relation openapi:serverInfo with servInd
29:    parameterObjects ← parameters property               ▷ from Operation object
30:    merge parameterObjects with pathParameterObjects
31:    for parameter in parameterObjects do
32:        parseParameterObject(parameter, opInd, schemaList)
33:    reqBodyInd = parseRequestBodyObject(Request Body object, schemaList)  ▷
   Body individual
34:    opInd add relation openapi:requestBody with reqBodyInd
35:    for Response object in responses property do         ▷ from Operation object
36:        respInd = parseResponseObject(statusCode, response object, schemaList)  ▷
   Response individual
37:        opInd add relation openapi:response with respInd
```

Algorithm 5 Handling of parameter object

```
1: function parseParameterObject(Parameter object, Operation individual,
   schemaList)                                  ▷ schemaList, List of Schema objects
2:     type ← parameter get property in
3:     if type is query then
4:         queryInd = parseQueryParameterObject(parameter, schemaList)   ▷
   QueryParam individual
5:         individual add relation openapi:parameter with queryInd
6:     else if type is path then
7:         pathParamInd = parsePathParameterObject(parameter, schemaList)   ▷
   PathParam individual
8:         opInd add relation openapi:parameter with pathParamInd
9:     else if type is cookie then
10:        cookieInd = parseCookieParameterObject(parameter, schemaList)   ▷ Cookie
   individual
11:        opInd add relation openapi:cookie with cookieInd
12:    else if type is header then
13:        headerInd = parseHeaderParameterObject(parameter name, parameter,
   schemaList)                                                  ▷ Header individual
14:        opInd add relation openapi:requestHeader with headerInd
```

Algorithm 6 Handling of Request Body and Response objects

```
1: function parseRequestBodyObject(Request Body object, schemaList)
2:     reqBodyInd ← create Body individual
3:     mediaTypeList ← from property content
4:     for mediaType in mediaTypeList do
5:         mediaTypeInd = parseMediaTypeObject(mediaType name, mediaType,
   schemaList)
6:         reqBodyInd add relation openapi:content with mediaTypeInd
       return reqBodyInd
7: function parseResponseObject(statusCode, Response object, schemaList)
8:     respInd ← create Response individual          ▷ based on status code family
9:     respHeaders ← from property headers
10:    for headerObj in respHeaders do
11:        headerInd = parseHeaderParameterObject(header name, headerObj, schema-
   List)
12:        respInd add relation openapi:responseHeader with headerInd
13:    respMediaTypes ← from property content
14:    for mediaType in respMediaTypes do
15:        mediaTypeInd = parseMediaTypeObject(mediaType name, mediaType,
   schemaList)
16:        respInd add relation openapi:content with mediaTypeInd
       return respInd
```

Table 2. Mapping Schema properties to SHACL.

Schema Object Property	SHACL Property
allOf	sh:and
oneOf	sh:xone
anyOf	sh:or
not	sh:not

function *parseMediaTypeObject* to create the individual and then associate it with the *Body* individual. Depending on the value *statusCode* of the *Response* object, function *parseResponseObject* creates an individual of the corresponding class. To map the properties of the *Response* object two lists are created, one for the *Header* individuals and one for the *Media Type* individuals which are handled by functions *parseHeaderObject* and *parseMediaTypeObject* respectively. The list of *Header* individuals is mapped to *responseHeader* property of the *Response* individual. The list of *Media Type* individuals is mapped to the content property of the *Response* individual.

4.2. *Instantiating Schema objects*

Schema objects are often defined under the *Components* section of an OpenAPI document and can be referenced by every other object. *Schema* objects are mapped to classes using SHACL. SHACL can be used to describe and validate the structure of RDF data, similar to XML-Schema or JSON Schema. It provides an RDF vocabulary of built-in types of constraints (e.g. cardinality: *minCount*, *maxCount*). Table 2 shows also how the *allOf*, *oneOf*, *anyOf* and *not* JSON Schema properties are expressed in SHACL. This allows for expressing *Schema* objects and their classes as compositions of existing ones. Polymorphism allows different views on objects (i.e. a *Schema* object that accepts alternative datatypes).

Each *Schema* object is mapped to a Shape class using SHACL. The Shape Class is distinguished into NodeShape class and PropertyShape class. The NodeShape class represents the classes that describe the models of an OpenAPI service description and, PropertyShape class represents the properties of a class, their datatype(s), and restrictions. A Shape class determines how to validate a focus node (a node from the data graph) based on the values of properties and other characteristics of the focus node. The two types of Shape classes are defined in SHACL. A NodeShape contains targets that specify which nodes in the data graph must conform to a Shape and constraint components that determine how to validate a node. Except for *sh:targetNode*, which specifies directly the nodes to be validated, there is also a *sh:targetClass* to denote that all the nodes of a given type need to conform with a particular Shape. Finally, *sh:path* of a *PropertyShape* points at the URI of the property that is being restricted.

Algorithm 7 Handling of Schema objects

```
1: function parseSchemaObject(schema name, Schema object, schemaList)
2:     if Schema object has one of the properties allOf, oneOf, anyOf then
3:         shape = createNodeShape(schema name, Schema object, schemaList)
4:         innerSchemas ← get Schema objects from properties allOf, anyOf, oneOf
5:         for schema in innerSchemas do
6:             innerShape = parseSchemaObject(schema name, shchema, schemaList)
7:             shape add relation with innerShape          ▷ based on mapping of Table 2
           return shape
8:     else
9:         schemaType ← from property type
10:        if schemaType = object then
11:            shape = createNodeShape(schema name, Schema object, schemaList)
12:            return shape
13:        else if schemaType = int or boolean or string then
14:            shape = createPropertyShape(empty, schema name, schema object,
   schemaList)
15:            return shape
16:        else if schemaType = array then
17:            shape = createCollectionNodeShape(schema name, Schema object,
   schemaList)
18:            return shape
```

Algorithm 7 converts *Schema* objects to Shapes. Function *createNodeShape* generates the NodeShape from the given *Schema* object and also checks for any usage of extension properties and treats them respectively. For every property of the *Schema* object, function *createPropertyShape* creates the respective *PropertyShape*, that will be linked to the NodeShape through property *sh:property*. Function *createPropertyShape* is responsible for defining *PropertyShape* individuals. The function will define the *PropertyShape* and for any extension property, it will associate a semantic concept to the Shape. Function *createCollectionNodeShape* handles *Schema* objects of type array. The function generates a NodeShape associated semantically with a subclass of the OpenAPI ontology Collection class. Then, it calls function *createPropertyShape* for the *Schema* objects defined under the item property and maps them with the created *NodeShape individual*. The mapping of *Schema* objects to Shapes is described in the author's thesis.[4]

Listing 7 shows the model of a Pet with property *id*. Listing 8 is the corresponding Shape class. The model becomes an instance of the NodeShape class with the target class Pet. The *rdfs:label* predicate is human-readable information. The Shape node has one property (i.e. *Pet_idPropertyShape*) of type *PropertyShape* of type *Pet_id* which is the class that is being restricted.

Listing 8 shows that each *sh:property* becomes an instance of PropertyShape class. The values of *sh:datatype* come from the type and format of each property

Listing 7. Pet object example.

```
Pet:
type: object
required:
 - id
properties:
 id:
  type: integer
  format: int64
```

Listing 8. Shape and Property Shape nodes of Pet object.

```
<PetNodeShape> a sh:NodeShape ;
rdfs:label "PetNodeShape" ;
sh:property <Pet_idPropertyShape> ;
sh:targetClass <Pet> .

<Pet> a owl:Class .

<Pet_idPropertyShape> a sh:PropertyShape ;
rdfs:label "Pet_idPropertyShape" ;
openapi:name "id" ;
sh:datatype xsd:long ;
sh:path <Pet_id> .

<Pet_id> a rdf:Property .
```

Listing 9. Annotated Pet object.

```
Pet:
x-refersTo: https://example.com/ontology/Pet
type: object
required:
   - id
properties:
 id:
  x-refersTo: https://example.com/ontology/Id
  type: integer
  format: int64
```

Listing 10. Shape models with *x-refersTo* properties.

```
<PetNodeShape> a sh:NodeShape ;
rdfs:label "PetNodeShape" ;
sh:property <Pet_idPropertyShape> ;
sh:targetClass <https://example.com/ontology/Pet>.

<Pet_idPropertyShape> a sh:PropertyShape ;
rdfs:label "Pet_idPropertyShape" ;
openapi:name "id" ;
sh:datatype xsd:long ;
sh:path <https://example.com/ontology/Id>.
```

(i.e. *int32* has value *xsd:int* and *int64* have value *xsd:long*). The *sh:path* predicate points at the URI of the property that is being restricted.

In Listing 9 both, the Pet *Schema* object and its property are annotated. The *x-refersTo* property associates the object and its property to a semantic value (class) that becomes the target and path objects of their corresponding Shapes. This is showcased in Listing 10. If the extension property is *x-kindOf* the target and path objects become sub-classes of the designated models. Listing 10 must be modified according to Listing 11. The *x-mapsTo* annotation property can also be used in a *Schema* object to designate that it points to another *Schema* object or property

Listing 11. Shape models with *x-kindOf* properties.

```
<PetNodeShape> a sh:NodeShape ;
rdfs:label "PetNodeShape" ;
sh:property <Pet_idPropertyShape> ;
sh:targetClass <https://example.com/ontology/Pet>.

<Pet> a owl:Class ;
rdfs:subClassOf <https://example.com/ontology/Pet> .

<Pet_idPropertyShape> a sh:PropertyShape ;
rdfs:label "Pet_idPropertyShape" ;
openapi:name "id" ;
sh:datatype xsd:long ;
sh:path <https://example.com/ontology/Id>.

<Pet_id> a rdf:Property ;
rdfs:subPropertyOf <https://example.com/ontology/Id>.
```

Listing 12. A collection of Pet objects.

```
PetCollection:
 x-collectionOn: pets
 type: object
 properties:
  pets:
   type: array
    items:
     $ref: '#/components/schemas/Pet'
```

Listing 13. Shape models for a collection of Pet objects.

```
<PetCollectionNodeShape> a  sh:NodeShape ;
rdfs:label "PetCollectionNodeShape";
sh:property  <PetCollection_PropertyShape>;
sh:targetClass <PetCollection> .

<PetCollection> a   owl:Class ;
rdfs:subClassOf openapi:Collection.

<PetCollection_petsPropertyShape> a sh:PropertyShape ;
rdfs:label "PetCollection_petsPropertyShape";
openapi:name "pets" ;
sh:node <PetNodeShape> ;
sh:path openapi:member .
```

Schema with the same semantics. It is handled similarly to *x-refersTo* but the target and path objects now link to the model that the *x-mapsTo* property refers to.

In Listing 12, the *x-collectionOn* property indicates that a *Schema* object is a collection. Collections are represented using *openapi:Collection* class. The target class is PetCollection and will become a subclass of the Collection class. The Shape Property class corresponds to property *pets* and declares that each member of the collection is a pet. This is showcased in Listing 13.

Composition: *Schema* objects can also be defined using any of the keywords *allOf*, *anyOf*, *oneOf*, and *not*. The *allOf* is used to define new *Schemas* as a composition of the *Schemas* referred to after the keyword. The *allOf* keyword expresses the concept of inheritance: the resulting composite schema inherits the properties of all referred *Schemas*. Keywords *oneOf* and *anyOf* are used to express the concept of polymorphism: they are used to define new *Schemas* that can take the form of one

Listing 14. Object composition with *allOf*.

```
Pet:
allOf:
- $ref: '#/components/schemas/OldPet'
- type: object
required:
- id
properties:
 id:
  type: integer
  format: int64

OldPet:
 type: object
  required:
  - name
  - tag
properties:
 name:
  type: string
 tag:
  type: string
```

Listing 15. Composed Shape models.

```
<PetNodeShape> a  sh:NodeShape;
rdfs:label  "PetNodeShape" ;
sh:and  ( [ a sh:NodeShape;
 sh:property  <Pet_idPropertyShape>
]
<OldPetNodeShape>
);
sh:targetClass  <Pet>

<OldPetNodeShape> a  sh:NodeShape  ;
rdfs:label    "OldPetNodeShape" ;
sh:property  <OldPet_tagPropertyShape>, <OldPet_namePropertyShape
    >;
sh:targetClass  <OldPet>

<Pet>  a   owl:Class ;
rdfs:subClassOf  <OldPet> .

<OldPet>  a  owl:Class .
```

or more alternative *Schemas*. Finally, keyword *not* is used to restrict a *Schema* (i.e. it declares which type of value is not accepted as value for an object property).

The Pet *Schema* object of Listing 14 extends the OldPet with the property *id*. The PetNodeShape is defined using SHACL predicate *sh:and* to express that it accepts the properties of the focus node OldPetNodeShape (i.e. the Shape of the node being extended) and the additional Shape property of a blank node. The new class must inherit the properties of the original class. The way to express this is to define PetNodeShape as a sub-class of OldPetNodeShape class.

The addition of semantic annotations in composed *Schema* objects is translated to inheritance relations between semantic models. For example, the extension property *x-refersTo: https://example.com/ontology/Pet* is added in Listing 14 before the *allOf* keyword. The algorithm will create a class with the name of the model referred to. This will become the target class of the new model (i.e. *sh:targetClass* <*https://example.com/ontology/Pet*> will replace line 8 in Listing 15). Besides, the OldPet *Schema* can be annotated as well by adding an extension property *x-refersTo: https://example.com/ontology/OldPet* (i.e. between lines 12 and 13 in Listing 14). Then, the target class of the object being extended will be the class

Listing 16. Annotated composed Shape models.

```
1  <PetNodeShape>
2
3  a sh:NodeShape ;
4  rdfs:label      "PetNodeShape" ;
5  sh:and ( [ a  sh:NodeShape ;
6  sh:property  <Pet_idPropertyShape>
7  ]
8   <OldPetNodeShape>
9  ) ;
10 sh:targetClass  <https://example.com/ontology/Pet> .
11
12 <https://example.com/ontology/Pet>
13 a  owl:Class ;
14 rdfs:subClassOf  <https://example.com/ontology/OldPet> .
15
16 <OldPetNodeShape>
17 a   sh:NodeShape ;
18 rdfs:label  "OldPetNodeShape" ;
19 sh:property <OldPet_tagPropertyShape>,
20 <OldPet_namePropertyShape> ;
21 sh:targetClass  <https://example.com/ontology/OldPet> .
```

referred to in the extension property (i.e. line *sh:targetClass <https://example.com/ontology/OldPet>* will replace line 13 in Listing 15). Also, class < *OldPet* > in line 20 will not be created. In line 18, the model *http://example.com/ontology/OldPet* will become a superclass. Listing 16 shows the resulting composed Shape class when both models are annotated. There is also a case where an annotation property (i.e. for Pet or OldPet) has value *none*. This has the following impact on the instantiation process: (a) no target class will be created for any model annotated with *none*, (b) the Shape models for both, Pet and OldPet will be created but, (c) there will be no sub-class relationship between them.

Polymorphism: Often API requests and responses are described by alternative Schemas. In OpenAPI this can be expressed using keywords *oneOf* and *anyOf*. The Shape model is expressed using SHACL predicates *sh:xone* and *sh:or* respectively. Listing 17 shows an unnamed *Schema* for response with code "201" which can be one of Dog or Cat. Because the *Schema* has no name, the resulting Shape node of Listing 18 is a blank NodeShape (i.e. a Shape without a URI) with predicate *sh:xone* and an RDF list as object and member individuals CatNodeShape and DogNodeShape. The case with keyword *anyOf* is handled likewise; the only difference is that predicate *sh:xone* in line 3 of Listing 18 will be replaced by keyword *sh:or*.

Polymorphism in property *Schemas* is also supported. Properties can take alternative *Schemas*. Listing 19 shows that property *speed* can be of any type following keyword *anyOf* which, in turn, can be defined inline or referred to using keyword *$ref*. The resulting Shape model is shown in Listing 20 and is composed of one named Shape class (for the property referred to by *$ref* and two un-named classes for the remaining properties). Keyword *oneOf* in property *Schemas* are handled likewise by replacing predicate *sh:or* in line 4 of Listing 20 with predicate *sh:xone*.

A property *Schema* with polymorphism means that it accepts alternative datatypes. There are two cases. The first is when a semantic value is connected

Listing 17. Polymorphed un-named object.

```
responses:
 "201":
 description: variety
 content: application/json:
  schema:
   oneOf:
    -$ref: "#/components/schemas/Cat"
    -$ref: "#/components/schemas/Dog"

components:
 schemas:
 Dog:
  type: object
  properties:
   bark:
    type: boolean
   breed:
    type: string
 Cat:
  type: object
  properties:
   hunts:
    type: boolean
   age:
    type: integer
```

Listing 18. Polymorphed Shape object.

```
 openapi:schema  [ a  sh:NodeShape ;
 sh:xone  ( <DogNodeShape> <CatNodeShape> )
 ]

 <CatNodeShape>  a  sh:NodeShape ;
 rdfs:label  "CatNodeShape" ;
 sh:property <Cat_agePropertyShape>,<Cat_huntsPropertyShape>;
 sh:targetClass  <Cat> .

 <Cat>  a  owl:Class .

 <DogNodeShape>  a   sh:NodeShape ;
 rdfs:label  "DogNodeShape" ;
 sh:property <Dog_breedPropertyShape>,  <Dog_barkPropertyShape>;
 sh:targetClass  <Dog> .

 <Dog>  a   owl:Class .
```

Listing 19. Polymorphism in property Schemas.

```
 Pet:
 type: object
 required:
  - speed
 properties:
  speed:
  anyOf:
   - type: integer
     format: int64
 - type: string
 - $ref: '#/components/schemas/Specified'

 components:
  schemas:
  Specified:
   type: integer
   format: int32
 ...
```

with different *Schemas* through a property. The second is when a property is connected with semantic values through property attributes (i.e. properties of properties). Both, are introduced in property *Schemas* with polymorphism along with semantic annotations. When a property is annotated with a semantic value,

Listing 20. Polymorped property Shape model.

```
<Pet_speedPropertyShape>  a   sh:PropertyShape ;
rdfs:label    "Pet_speedPropertyShape" ;
openapi:name  "speed" ;
sh:or  ( <SpecifiedPropertyShape>
[ a  sh:PropertyShape ;
 sh:datatype  xsd:string
]
[ a sh:PropertyShape ;
 sh:datatype  xsd:long
]
) ;
sh:path   <Pet_speed> .
<Pet_speed> a rdf:Property .

<SpecifiedPropertyShape>  a  sh:PropertyShape ;
rdfs:label  "SpecifiedPropertyShape" ;
sh:datatype  xsd:int ;
sh:path      <Specified> .

<Specified>  a  rdf:Property .
...
```

Listing 21. Polymorphism with annotated property Schemas.

```
Pet:
type: object
required:
 - speed
properties:
 speed:
 anyOf:
  - type: integer
  format: int64
  x-refersTo: https://example.com/ontology/Int64_property
  - type: string
  x-refersTo: https://example.com/ontology/String_property
   - $ref: '#/components/schemas/Specified'

components:
 schemas:
  Specified:
  x-refersTo: https://example.com/ontology/Int32_property
  type: integer
  format: int32
 ...
```

the *Schema* property and the semantic value are connected. For example, if the annotation *x-refersTo: https://example.com/ontology/Speed* is inserted between lines 6 and 7 in Listing 19, the property can be of any listed after keyword *anyOf* (or after keyword *oneOf* in a similar example). The annotation is used to connect the *Schema* of property *speed* with the *Schemas* in the RDF list in Listing 20. The second case is when the property attributes are semantically annotated instead of the property itself. As shown in Listing 21, if any of the properties following keyword *anyOf* (equivalently *oneOf*) is annotated, property *speed* does not acquire a semantic value but its attributes do. In Listing 22, the Shape model of *speed* is presented along with the semantic value of each of the nodes. There are also cases that are invalid (i.e. may create inconsistencies).[4]

Restriction: Keyword *not* is used to express not acceptable values (i.e. it makes *Schemas* more specific). In Listing 23, property *byType* can be anything but a string. The SHACL constraint is expressed with predicate *sh:not*. The Shape model

Listing 22. Polymorphed Shape models with annotated property Shape Schemas.

```
Pet_speedPropertyShape> a  sh:PropertyShape ;
rdfs:label    "Pet_speedPropertyShape" ;
openapi:name  "speed" ;
sh:or  ( <SpecifiedPropertyShape>
[ a  sh:PropertyShape ;
 sh:datatype  xsd:string ;
 sh:path <https://example.com/ontology/String_property>
]
[ a  sh:PropertyShape ;
 sh:datatype  xsd:long ;
 sh:path <https://example.com/ontology/Int64_property>
]
) .

<SpecifiedPropertyShape> a sh:PropertyShape ;
rdfs:label   "SpecifiedPropertyShape" ;
sh:datatype  xsd:int ;
sh:path <https://example.com/ontology/Int32_property> .
...
```

Listing 23. Restricted property Schema.

```
Pet:
type: object
required:
- id
properties:
 id:
  type: integer
  format: int64
 byType:
  not:
   type: string
```

Listing 24. Shape of restricted property Schema.

```
<Pet_byTypePropertyShape>  a  sh:PropertyShape ;
rdfs:label    "Pet_byTypePropertyShape" ;
openapi:name  "byType" ;
sh:not [ a  sh:PropertyShape ;
 sh:datatype  xsd:string
] ;
sh:path  <Pet_byType> .

<Pet_byType>  a  rdf:Property .
```

in Listing 24 denotes that the corresponding *PropertyShape* may not take any of the values specified in the RDF list. The restricted property cannot be annotated (e.g. by adding *x-refersTo: https://example.com/ontology/SpecificString* after line 10 in Listing 23). This would allow every other string but the one referred to. This negates the universality of the *not* keyword and its original purpose.

5. Ontology Evaluation

Ontology evaluation methods[13] investigate the extent to which an ontology conforms to the criteria of accuracy, completeness, conciseness, adaptability, clarity, and consistency. The OpenAPI ontology conforms to the above standards to the extent that OpenAPI specification does. It supports almost the entire range of

OpenAPI objects and the properties foreseen in the specification, with the exception of *Webhooks*, *Links*, and *Callbacks* (i.e. left for future work). Evaluating a complex qualitative ontology is not easy. A detailed evaluation of the OpenAPI ontology would require a team of expert users to run an exhaustive set of tests (also left as future work). Task-based ontology evaluation approaches consider also the criterion of computational efficiency: An ontology is evaluated based on its performance in a specific task. In the following, the evaluation of the OpenAPI ontology is mainly focused on the efficiency of ontology translation and query searching.

The Web application[h] supports uploading OpenAPI descriptions of REST services (in YAML or JSON format) and their instantiation to the ontology. It comprises three main services. The front-end service provides a simple user interface for the user to interact with the service. The semantic service acts as the middle layer between the front end and the triple store. It is responsible for the conversion of service descriptions to ontologies and for the validation and execution of SPARQL queries. The triple store service is a Virtuoso Universal Server[i] acting as a database for persisting and querying the generated ontologies. It takes more than 1 second to translate an OpenAPI service description (i.e. YAML file with more than 200 lines) to ontology and store it in Virtuoso.

More than 300 OpenAPI descriptions have been downloaded from SwaggerHub[j] and translated to ontologies. The user may search the triple store using SPARQL. The SPARQL query of Listing 25 requests all services and operations related to concept *schema:Pet*. Because the reasoner is invoked prior to storage, the query will also retrieve services related to all its subclass concepts (i.e. *schema:Lizard*). Therefore, the OpenAPI ontology can be used for improving the operation of a search engine over a collection of OpenAPI descriptions (e.g. SwaggerHub).

The performance of queries depends also on query complexity and specificity. Table 3 illustrates the response time of two more indicative SPARQL queries and shows that the response time increases as queries become more complicated.

Listing 25. Example query on hidden relationships.

```
PREFIX openapi: <http://www.intelligence.tuc.gr/ns/open-api>
PREFIX product : <http://www.productontology.org/doc/>

SELECT ?serviceName ?operationId
WHERE {
 ?document a openapi:Document .
 ?document openapi:info ?serviceInfo .
 ?serviceInfo openapi:serviceTitle ?serviceName .
 ?document openapi:supportedEntity ?schemaInd .
 ?schemaInd openapi:targetClass product:Pet.
 ?schemaInd openapi:supportedOperation ?operationInd .
 ?operationInd openapi:operationId ?operationId .
}
```

[h]https://www.intelligence.tuc.gr/semantic-open-api/
[i]https://virtuoso.openlinksw.com
[j]https://app.swaggerhub.com/

Table 3. Response time of SPARQL queries on the OpenAPI ontology data set.

Simple Query	Complex Query

SELECT ?servTitle ?code ?opName
WHERE {
?service a openapi:Document .
?service openapi:info ?info .
?info openapi:serviceTitle ?servTitle .
?service openapi:supportedOperation ?operation .
?operation openapi:response ?response .
?response openapi:statusCode ?code .
?operation a openapi:Operation .
?operation openapi:name ?opName .
}
	SELECT ?servTitle ?code ?opName
	WHERE {
	?service a openapi:Document .
	?service openapi:info ?info .
	?info openapi:serviceTitle ?servTitle .
	?service openapi:supportedOperation ?operation .
	?operation openapi:response ?response .
	?response openapi:content ?recon .
	?recon openapi:schema ?shape .
	?shape a sh:NodeShape .
	{ ?shape sh:targetClass purl:Pet . }
	UNION { ?shape sh:targetClass purl:Pet . }
	OPTIONAL {
	?tag a openapi:Tag .
	?tag openapi:name ?tgName . }
	OPTIONAL {
	?service openapi:supportedOperation ?operation .
	?operation openapi:serverInfo ?server .
	?server a openapi:Server .
	?server openapi:host ?serverUrl . }
	}

Get all operations for every service as well as the described status codes	Get all services with operations having responses referring on specific semantic concepts
Response time: 2193 ms	Response time: 8341 ms

6. Future Work

Query formulation can be facilitated by a query-by-example interface allowing users to select service properties from a pull-down menu. A dedicated query language in the spirit of SOWL-QL[14] will be designed so that the user need not be familiar with the peculiarities and syntax of the underlying ontology. Query optimization and indexing on common properties will also improve performance. Enabling automatic service synthesis is a long-term and more ambitious goal. The ontology is being extended to support the translation of Links and Callbacks OpenAPI properties which are introduced in the latest update of the specification. Callbacks are asynchronous requests that the server will send to some other service in response to certain events. This feature improves the workflow that the server API offers to its clients. Links, enable the description of how various values returned by one operation can be used as input for other operations. Both these new features will bring more support to HATEOAS in the ontology.

References

1. A. Karavisileiou, N. Mainas and E. G. Petrakis, Ontology for OpenAPI REST services descriptions, in *IEEE Int. Conf. on Tools with Artificial Intelligence* (*ICTAI 2020*) (Virtual Conference, 2020), pp. 25–40.
2. M. Lanthaler and C. Gütl, A vocabulary for hypermedia-driven Web APIs, in *Workshop on Linked Data on the Web* (*LDOW 2013*) (Rio de Janeiro, Brazil, 2013).
3. H. Knublauch and D. Kontokostas, Shapes Constraint Language (SHACL) (July 2017).
4. F. Bouraimis, Instantiating OpenAPI Descriptions to the REST Services Ontology, Tech. Rep., Diploma Thesis, School of Electrical and Computer Engineering, Technical University of Crete (TUC) (Chania, Crete, 2021).
5. R. Chinnici, J.-J. Moreau, A. Ryman and S. Weerawarana, Web Services Description Language (WSDL) Version 2.0 Part 1: Core Language (June 2007), W3C Recommendation.
6. N. Mitra and Y. Lafon, SOAP Version 1.2 Part 0: Primer (Second edition) (April 2007).
7. J. Farrel and H. Lausen, Semantic Annotations for WSDL and XML Schema (August 2007), W3C Recommendation.
8. M. Hadley, Web Application Description Language (August 2009), W3C Recommendation.
9. D. Martin *et al.*, OWL-S: Semantic Markup for Web Services (November 2004), W3C Member Submission.
10. A. Sheth, K. Gomadam and J. Lathem, SA-REST: Semantically interoperable and easier-to-use services and mashups, *IEEE Internet Computing* **11** (2007) 91–94.
11. F. A. Musyaffa, L. Halilaj, R. Siebes, F. Orlandi and S. Auer, Minimally invasive semantification of light weight service descriptions, in *IEEE Int. Conf. on Web Services* (*ICWS 2016*) (San Francisco, CA, USA, 2016), pp. 672–677.
12. S. Schwichtenberg, C. Gerth and G. Engels, From Open API to semantic specifications and code adapters, in *IEEE Int. Conf. on Web Services* (*ICWS 2017*) (San Francisco, CA, USA, 2017), pp. 484–491.
13. J. Raad and C. Cruz, A survey on ontology evaluation methods, in *Int. Joint Conf. on Knowledge Discovery, Knowledge Engineering and Knowledge Management* (*IC3K 2015*) (Lisbonne, Portugal, 2015), pp. 179–186.
14. K. Stravoskoufos, E. Petrakis, N. Mainas, S. Batsakis and V. Samoladas, SOWL QL: Querying spatio-temporal ontologies in OWL, *Journal on Data Semantics* **5** (2016) 249–269.

Copyright of International Journal on Artificial Intelligence Tools is the property of World Scientific Publishing Company and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.